Unsupervised Learning



© 2025 Olivier Claerbout

Lesmateriaal

- Via GitHub account
- © 2025 Olivier Claerbout

  Dit materiaal mag niet worden gekopieerd, verspreid, gepubliceerd of anderszins gereproduceerd zonder uitdrukkelijke schriftelijke toestemming van de auteur. Dit geldt eveneens voor de Jupyter Notebooks!

© 2025 Olivier Claerbout

Overzicht

- Theorie en Intuïtie van PCA.
- Handmatig een PCA algoritme creëren.
- Gebruik Scikit-Learn om PCA uit te voeren.
- Overzicht van PCA Oefenproject.
- Oplossing voor PCA Oefening.

© 2025 *Olivier Claerbout*

Principal Components Analysis

© 2025 *Olivier Claerbout*

PCA

- Voorlopig hebben we ons inzake unsupervised learning geconcentreerd op clustertechnieken, die proberen labels te "ontdekken" voor gegevens met kenmerken die geen labels hebben.
- We zullen nu overgaan naar unsupervised algoritmen die zich richten op dimensiereductie.

© 2025 Olivier Claerbout

PCA

- Waarom is dit nuttig?
  - Stel je een dataset voor met meer dan 30 features. Hoe zou je de belangrijkste features ontdekken / correct begrijpen?
  - Visualisatie en Data Analyse hebben beperkingen wanneer het aantal features (dimensies) toeneemt.

© 2025 Olivier Claerbout

PCA

- Resultaten van Dimensiereductie:
  - Begrijp welke features de meeste variantie in de dataset veroorzaken.
  - Hulp bij het menselijk begrip van grote datasets met veel features, vooral door middel van visualisatie.
  - Een eenvoudiger dataset (minder features) kan worden gebruikt om ML algoritmes te trainen
    - Train ML algoritme op een kleinere dataset via PCA!

© 2025 Olivier Claerbout

PCA

- Resultaten van Dimensiereductie:
  - Gaat van N features naar een gewenste kleinere set via een transformatie
- Belangrijke opmerking:
  - Dimensiereductie algoritmes zoals PCA kiezen *niet simpelweg een subset van de bestaande features*.
  - Ze creëren nieuwe dimensionale componenten die combinaties zijn van proporties van de bestaande features.
  - Hiervoor worden eigenvectoren en eigenvalues gebruikt

© 2025 Olivier Claerbout

PCA

- Variantie die wordt verklaard door een feature in de dataset:
  - Sommige features hebben meer belang dan andere
  - Vb: de grootte van een woning is wellicht belangrijker dan de kleur ervan bij de bepaling van de waarde van de woning

Dit is eenvoudig te begrijpen wanneer we zien dat bepaalde features een sterkere correlatie hebben met een te voorspellen label dan andere.

*Maar wat met **ongelabelde data?***

Welke maatstaf voor belangrijkheid v/e feature kunnen we gebruiken?

© 2025 Olivier Claerbout

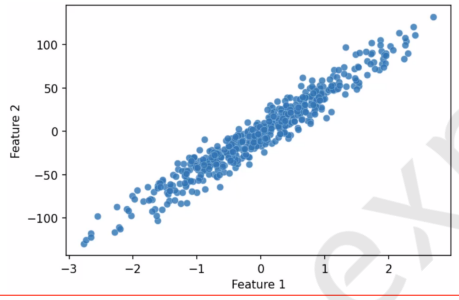
PCA

- "Verklaarde variantie"
  - Meet de mate waarin elke feature verantwoordelijk is voor de spreiding in de dataset
  - Features met hogere spreiding verklaren wellicht meer van de variantie in de dataset

© 2025 Olivier Claerbout

PCA

- “Verklaarde variantie”
  - Is er een manier om naar deze gegevens te kijken langs een andere dimensie die misschien features 1 & 2 combineert waardoor we minder data krijgen maar toch een groot deel van de oorspronkelijke variantie blijven verklaren?

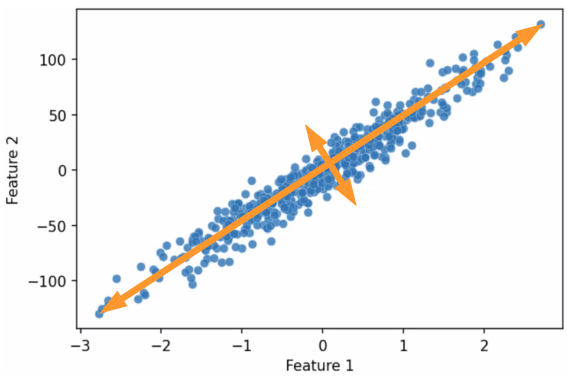


© 2025 Olivier Claerbout

PCA

- “Verklaarde variantie”
  - Met een lineaire combinatie van Feature 1 en Feature 2 kunnen we een lijn tekenen
  - We kunnen ook loodrecht op deze lijn een as tekenen

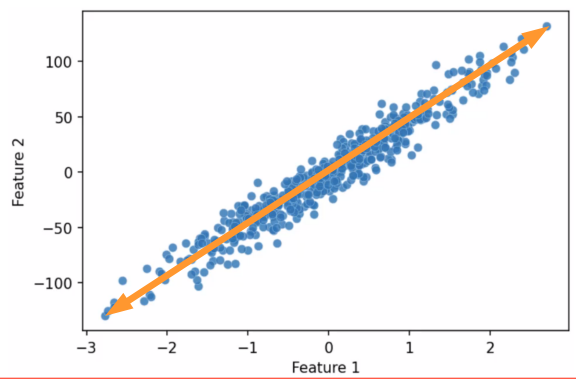
Welke “nieuwe as” verklaart de meeste spreiding in de gegevens?

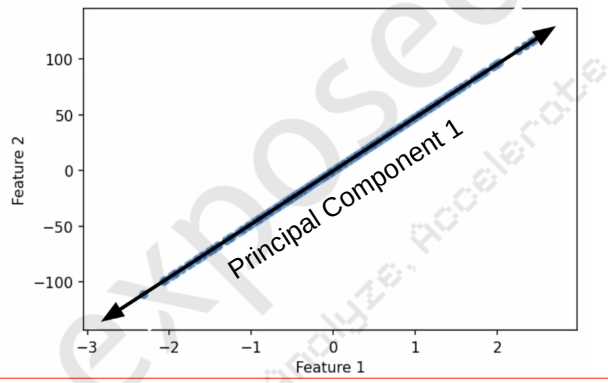


© 2025 Olivier Claerbout

PCA

- Als we van twee naar één dimensie willen gaan, zouden we deze nieuwe component kunnen gebruiken. Deze component is een lineaire combinatie van de features.
  - We projecteren ahw alle punten op die ene as die de meeste variantie verklaart.

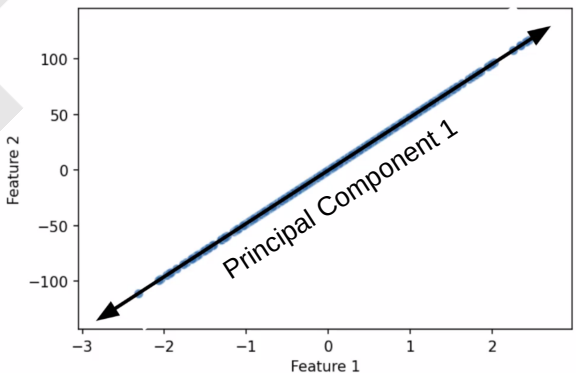




© 2025 Olivier Claerbout

PCA

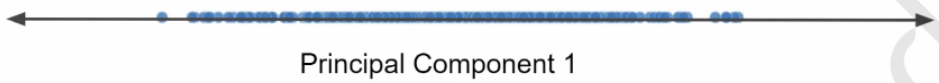
- PCA zal de belangrijkste componenten rangschikken in functie van de verklaarde variantie.
- We reduceren hier dus 2 features tot 1 hoofdcomponent.



© 2025 Olivier Claerbout

PCA

- We kunnen dit ook als volgt uitzetten:



- We kunnen dit doen van gelijk welk aantal features naar een beperkter aantal hoofdcomponenten, vb van 30 naar 5.

© 2025 Olivier Claerbout

PCA

- "Verklaarde variantie"
  - Hoofdcomponenten of Principal Components zijn een lineaire combinatie van originele features
  - Hoe meer variantie een originele feature verklaart, hoe meer invloed deze heeft op de door PCA ontdekte componenten

© 2025 Olivier Claerbout

PCA

- "Verklaarde variantie"
  - In het voorbeeld zijn we van 2 features naar 1 hoofdcomponent gegaan.
  - Deze hoofdcomponent op zich kan een deel van de variantie van de originele dataset verklaren, vb 90%
  - 100% van de variantie in de data wordt verklaard door alle originele features samen
  - We ruilen een deel verklaarde variantie in om minder dimensies te bekomen
  - Dit kan een aanzienlijke besparing betekenen bij data sets met veel dimensies, maar slechts een beperkt aantal "sterke" features

© 2025 Olivier Claerbout

PCA

- Hoe werkt PCA?
  - PCA creëert een nieuwe set dimensies (de hoofdcomponenten) die genormaliseerde lineaire combinaties van de originele features zijn.
  - Stel een originele dataset met features x1, x2, … tot xp

    → De **eerste** principal component is een lineaire combinatie van deze features:

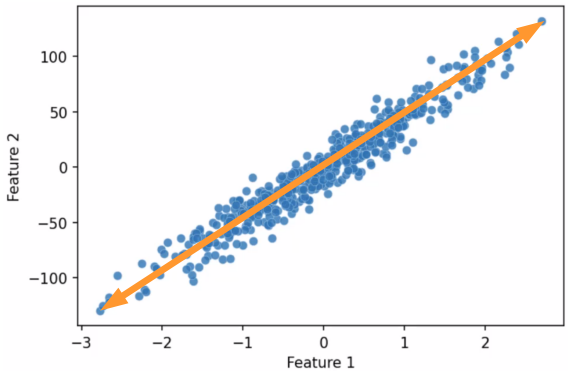
$$Z_1 = \phi_{11}x_1 + \phi_{21}x_2 + \ldots + \phi_{p1}x_p$$

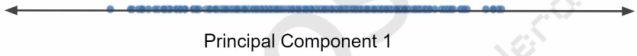
© 2025 Olivier Claerbout

PCA

- Hoe werkt PCA?

$$Z_1 = \phi_{11}x_1 + \phi_{21}x_2$$

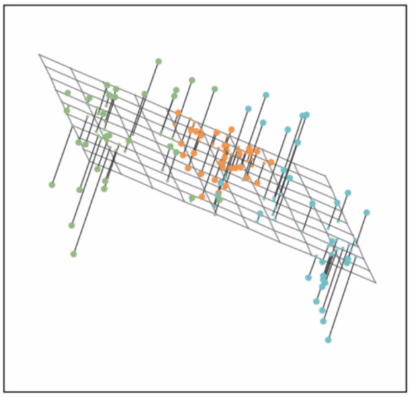


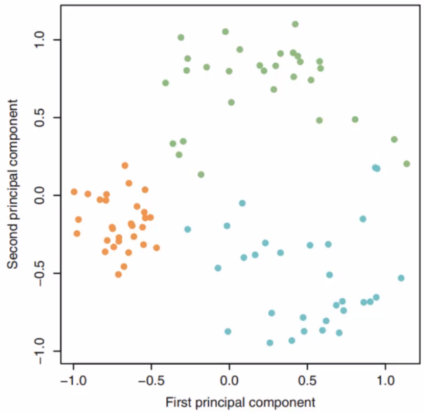


© 2025 Olivier Claerbout

PCA

- 3 dimensies:
  - PCA 1 & 2 vormen een vlak
  - Projecteer punten op vlak
  - 3 → 2 dimensies



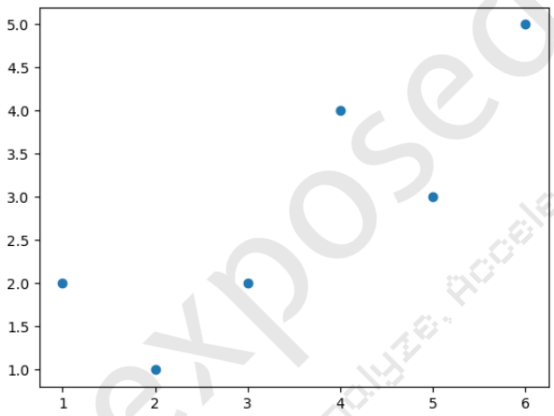


© 2025 Olivier Claerbout

PCA

- Hoe berekenen we deze componenten?
- We starten met een 2d dataset:

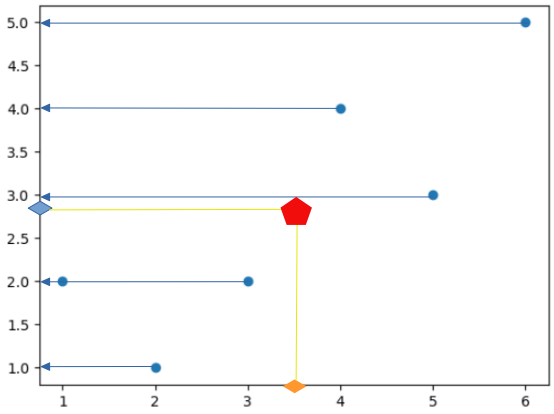
x1	x2
1	2
2	1
3	2
4	4
5	3
6	5



© 2025 Olivier Claerbout

PCA

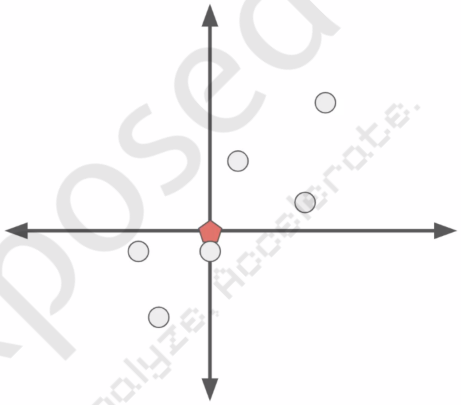
- De eerste stap is het standaardiseren van de data
- Eigenlijk verplaatsen we hiermee het gemiddelde van de dataset naar nul
  - Bereken voor elke feature het gemiddelde



© 2025 Olivier Claerbout

PCA

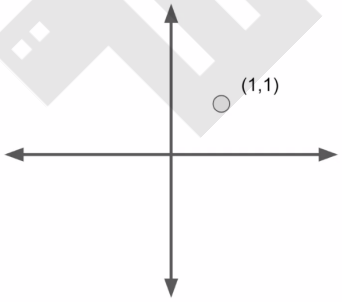
- We shiften de dataset naar een gemiddelde van nul (data is nu rond (0,0))
- De relatie tussen de punten is niet gewijzigd!
- Dat is wat standaardiseren doet!



© 2025 Olivier Claerbout

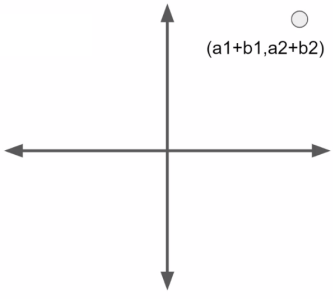
PCA

- We weten ondertussen: PCA is een lineaire transformatie van de feature space
- Lineaire transformatie van data
  - Stel een punt (1,1) in de originele feature space
  - Een lineaire transformatie is een vermenigvuldiging met een set waarden die we in een matrix kunnen voorstellen



(x,y) → (a1*x + b1*y, a2*x+b2*y)

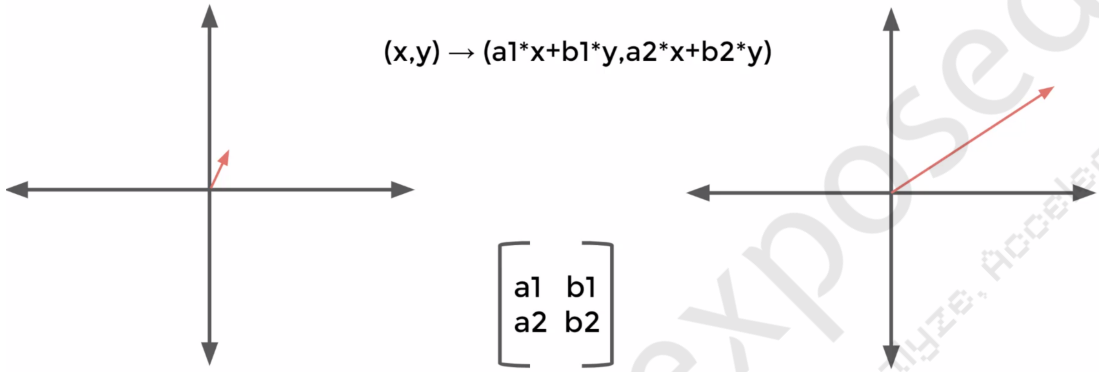
$$\begin{bmatrix} a1 & b1 \\ a2 & b2 \end{bmatrix}$$



© 2025 Olivier Claerbout

PCA

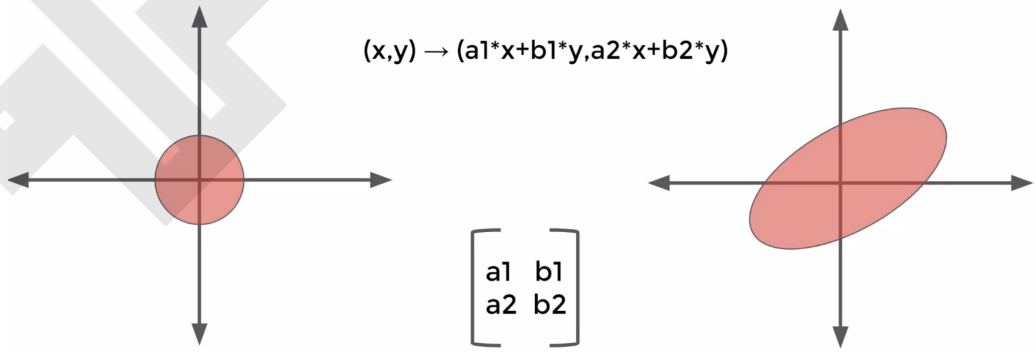
- Het voorgaande geldt eveneens voor vectoren
  - Na een lineaire transformatie kan een vector in een andere richting wijzen en een andere magnitude hebben



© 2025 Olivier Claerbout

PCA

- Dit kan ook voor een puntenvlak
  - Na een lineaire transformatie is dit vb “uitgerokken”

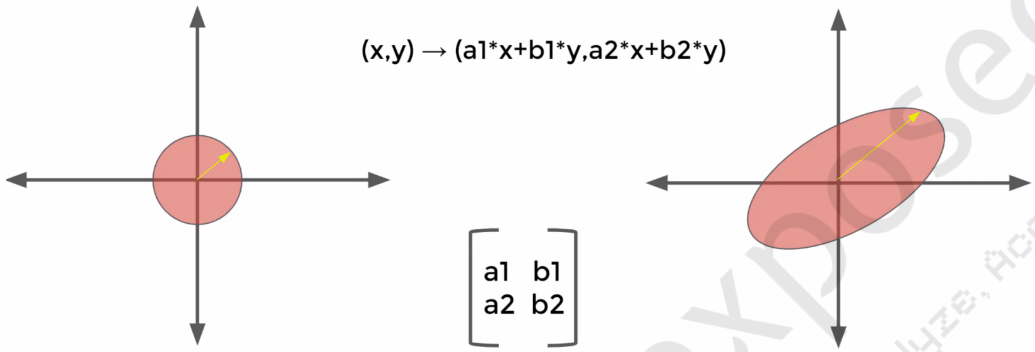


© 2025 Olivier Claerbout

Python exposed
Learn Python to Automate, Analyze, Accelerate.

PCA

- Belangrijk:
  - Bij deze lineaire transformaties is er een set vectoren voor elke dimensie waarvan de richting niet wijzigt bij de transformatie



© 2025 Olivier Claerbout

PCA

- Belangrijk:
  - Voor bijna alle vectoren geldt dat ze na een lineaire transformatie een andere richting hebben
  - Voor *elke dimensie* is er echter een vector waarvan de richting onveranderd zal blijven bij lineaire transformatie (de magnitude wijzigt wel)
  - Dit is een *Eigenvector*
  - Voor *elke feature dimensie* (x_1, x_2, ...) is er dus een Eigenvector die, na lineaire transformatie, overeenkomt met de Principal Component.
  - Een eigenvector is dus een vector die bij een lineaire transformatie alleen van grootte verandert, maar niet van richting
  - Voor elke eigenvector is er een orthogonale eigenvector

© 2025 Olivier Claerbout

python exposed
Learn Python to Automate, Analyze, Accelerate.

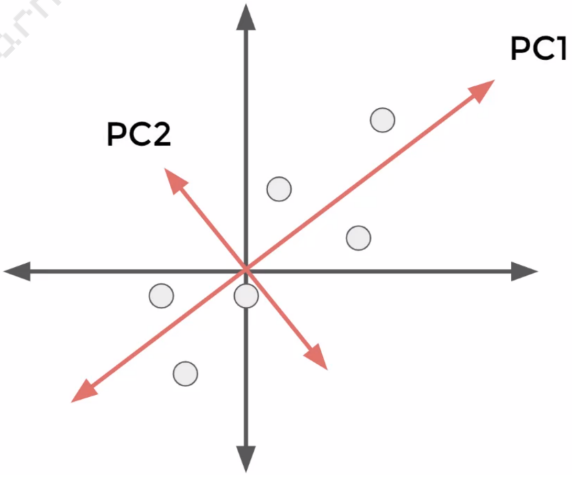
PCA

- PCA maakt gebruik van de covariantiematrix van de dataset om de richtingen (eigenvectoren) en variaties (eigenwaarden) te bepalen.
- De hoofdcomponenten (Principal Components) zijn de eigenvectoren van de covariantiematrix.
- Een eigenvector is een vector die na een lineaire transformatie dezelfde richting behoudt (maar mogelijk een andere grootte krijgt).
- Het aantal eigenvectoren is gelijk aan het aantal features in de dataset, en deze bepalen de richtingen van de Principal Components.
- De eigenwaarden bepalen de hoeveelheid variantie die door de bijbehorende eigenvector wordt verklaard.

© 2025 Olivier Claerbout

PCA

$$\begin{bmatrix} \text{Var}(X) & \text{Cov}(X,Y) \\ \text{Cov}(X,Y) & \text{Var}(Y) \end{bmatrix}$$



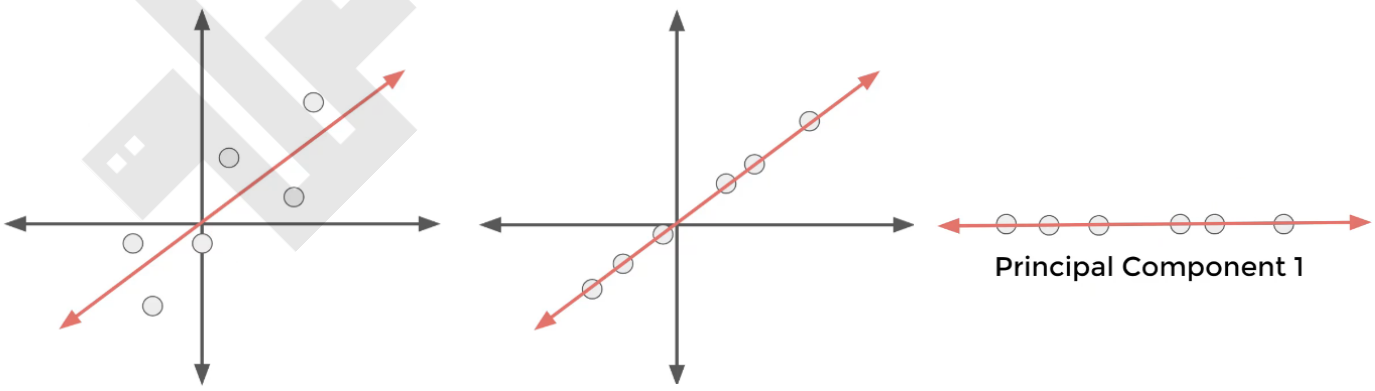
© 2025 Olivier Claerbout

PCA

- We hebben nu de Principal Components, die de belangrijkste variaties in de dataset beschrijven.
- De eigenwaarden geven aan hoeveel variantie elke Principal Component verklaart.
- We kiezen een subset van de belangrijkste Principal Components (eigenvectoren).
- De selectie gebeurt op basis van de eigenwaarden: we behouden de componenten met de hoogste waarden.
- Vervolgens projecteren we de originele datapunten op de geselecteerde eigenvectoren, waardoor we een nieuwe representatie van de data krijgen.
- Bijvoorbeeld, we reduceren van 2 features naar 1 Principal Component voor eenvoud en beter inzicht in de data.

© 2025 Olivier Claerbout

PCA



© 2025 Olivier Claerbout

PCA

- Stappen bij PCA:
  - Stap 1: Start met de originele dataset.
  - Stap 2: Bereken de covariantiematrix van de dataset.
  - Stap 3: Bereken de eigenwaarden en eigenvectoren van de covariantiematrix.
  - Stap 4: Sorteer de eigenvectoren op basis van hun bijbehorende eigenwaarden (van groot naar klein).
  - Stap 5: Selecteer de N grootste eigenwaarden en hun bijbehorende eigenvectoren.
  - Stap 6: Projecteer de originele data op de geselecteerde eigenvectoren om de nieuwe Principal Components te verkrijgen.

"Eigen" is een Duits woord wat typisch of karakteriserend betekent

© 2025 Olivier Claerbout

PCA

- Covariantie matrix: een vierkante matrix die de covariantie tussen elk paar dimensies in de dataset voorstelt.

$$\begin{bmatrix} \text{Var(X)} & \text{Cov(X,Y)} \\ \text{Cov(X,Y)} & \text{Var(Y)} \end{bmatrix}$$

$$Cov(x,y) = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{n-1}$$

$x = [1\ 2\ 3\ 4\ 5]$, $y = [3\ 5\ 7\ 7\ 9]$, $\bar{x} = 3$, $\bar{y} = 6,2$

	(x)	(y)	$(x-\bar{x})$	$(y-\bar{y})$	$(x-\bar{x})(y-\bar{y})$
	1	3	$1-3=-2$	$3-6,2=-3,2$	$(-2)\times(-3,2)=6,4$
	2	5	$2-3=-1$	$5-6,2=-1,2$	$(-1)\times(-1,2)=1,2$
	3	7	$3-3=0$	$7-6,2=0,8$	$0\times0,8=0$
	4	7	$4-3=1$	$7-6,2=0,8$	$1\times0,8=0,8$
	5	9	$5-3=2$	$9-6,2=2,8$	$2\times2,8=5,6$
$\sum$					14

$$\text{Steekproefcovariantie:} cov(x,y) = \frac{14}{5-1} = 3,5$$

$$\text{Populatiecovariantie:} cov(x,y) = \frac{14}{5} = 2,8$$

© 2025 Olivier Claerbout

PCA

- Berekening van de variantie:

  Voor elke observatie bereken je hoe ver deze van het gemiddelde af ligt, je kwadrateert die afwijking en vervolgens neem je het gemiddelde van al deze gekwadrateerde afwijkingen.

$$s^2 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n-1}$$

- Covariantie matrix:

$$\begin{bmatrix} \text{Var(X)} & \text{Cov(X,Y)} \\ \text{Cov(X,Y)} & \text{Var(Y)} \end{bmatrix}$$

© 2025 Olivier Claerbout



Bereken de EigenValues en de EigenVectoren van de Covariantie Matrix

© 2025 Olivier Claerbout

python exposed
Learn Python to Automate, Analyze, Accelerate.

PCA

- Voor een vierkante matrix A, maken een Eigenvector v en een Eigenvalue λ de volgende vergelijking waar:

  $Av = \lambda v$

  voorbeeld: voor deze matrix A $\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix}$

  is een Eigenvector v $\begin{bmatrix} 1 \\ 4 \end{bmatrix}$

  met als Eigenvalue λ = 6

© 2025 Olivier Claerbout

PCA

- Voor een vierkante matrix A, maken een Eigenvector v en een Eigenvalue λ de volgende vergelijking waar:

  $Av = \lambda v$

  Want: Av = $\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix}\begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} -6\times1+3\times4 \\ 4\times1+5\times4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$

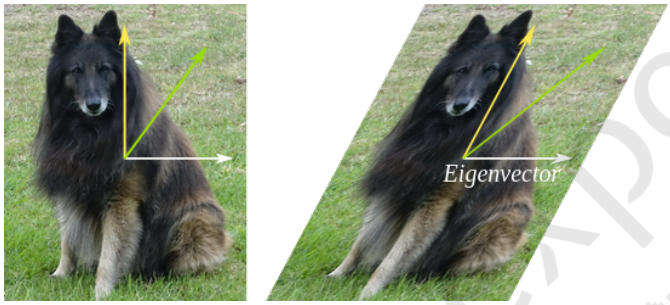
  En $\lambda v = 6 \times \begin{bmatrix} 1 \\ 4 \end{bmatrix} = \begin{bmatrix} 6 \\ 24 \end{bmatrix}$

© 2025 Olivier Claerbout

python exposed
Learn Python to Automate, Analyze, Accelerate.

PCA

- Merk op hoe we een matrix vermenigvuldigen met een vector en hetzelfde resultaat krijgen als door het vermenigvuldigen met een scalar!
- Een Eigenvector verandert niet van richting bij een transformatie:



© 2025 Olivier Claerbout

PCA

- Hoe vinden we deze Eigenvector en Eigenvalue voor een 2 × 2 matrix A?
  - Av = λv (v is een niet-mul vector; λ is een scalar)
    Av = λIv (I = identiteitsmatrix om matrix vs matrix te krijgen)
  - Av - λIv = 0 → (A - λI) v = 0 (een homogeen stelsel waarvoor de determinant van de matrix nul moet zijn)

    Als onze eigenvector v != 0 dan kunnen we dit oplossen voor λ gebruikmakend van de determinant:

$$A - \lambda I = 0$$

We hebben hier geen eigenvector in de vergelijking!

© 2025 Olivier Claerbout

python exposed
Learn Python to Automate, Analyze, Accelerate.

PCA

- Hoe vinden we deze Eigenvector en Eigenvalue voor een 2 × 2 matrix?

$$A - \lambda I = 0$$

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 0$$

$$\begin{bmatrix} -6-\lambda & 3 \\ 4 & 5-\lambda \end{bmatrix} = 0$$

De determinant van $\begin{bmatrix} -6-\lambda & 3 \\ 4 & 5-\lambda \end{bmatrix}$ = $(-6-\lambda)(5-\lambda) - (3)(4)$

© 2025 Olivier Claerbout

PCA

- Hoe vinden we deze Eigenvector en Eigenvalue voor een 2 × 2 matrix?

$|A - \lambda I| = 0$

$(-6-\lambda)(5-\lambda) - (3)(4) = 0$

$(-6-\lambda)(5-\lambda) - 12 = 0$

$-30 + 6\lambda - 5\lambda + \lambda^2 - 12 = 0$

$\lambda^2 + \lambda - 42 = 0$ *(een vierkantsvergelijking)*

© 2025 Olivier Claerbout

python exposed
Learn Python to Automate, Analyze, Accelerate.

PCA

- Terug in de tijd… onze vergelijking is $\lambda^2 + \lambda - 42 = 0$

Oplossen van een kwadratische vergelijking:

de discriminant $= b^2 - 4ac$
$= 1^2 - 4 \times 1 \times (-42)$
$= 169$

De discriminant D is positief dus er zijn twee verschillende reële oplossingen $x_1 \neq x_2$*:*

$$x_{1,2} = \frac{(-b \pm \sqrt{b^2 - 4ac})}{2a} \text{ofwel } x_{1,2} = \frac{(-b \pm \sqrt{D})}{2a}$$ dus (-1+13)/2 en (-1-13)/2 of 6 en -7

© 2025 Olivier Claerbout

PCA

- We hebben nu 2 eigenvalues 6 en -7. Hoe vinden we nu de eigenvectoren?
  Av = λv

  We stoppen daar nu de gekende waarde in:

$$\begin{bmatrix} -6 & 3 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = 6 \times \begin{bmatrix} x \\ y \end{bmatrix}$$

  Na vermenigvuldigen krijgen we:
  -6x + 3y = 0
  4x – 1y = 0
  → *Beide vergelijkingen geven y = 4x als oplossing dus de eigenvector is elk veelvoud van* $\begin{bmatrix} 1 \\ 4 \end{bmatrix}$

© 2025 Olivier Claerbout

Python exposed
Learn Python to Automate, Analyze, Accelerate.

PCA

- Stappen bij PCA:

  We hebben nu voor elke dimensie een eigenwaarde en bijbehorende eigenvector.

  - *Sorteer de eigenvectoren op basis van hun bijbehorende eigenwaarden (van hoog naar laag).*
  - *Selecteer de N grootste eigenwaarden en hun bijbehorende eigenvectoren, afhankelijk van het gewenste aantal dimensies.*
  - *Projecteer de originele data op de geselecteerde eigenvectoren om de nieuwe Principal Components te verkrijgen.*

© 2025 *Olivier Claerbout*

Principal Components Analysis

Jupyter notebooks:
00-PCA-Manuele-Implementatie.ipynb
01-PCA-in-Scikit-Learn.ipynb
02-PCA-Exercise-Project.ipynb

© 2025 *Olivier Claerbout*